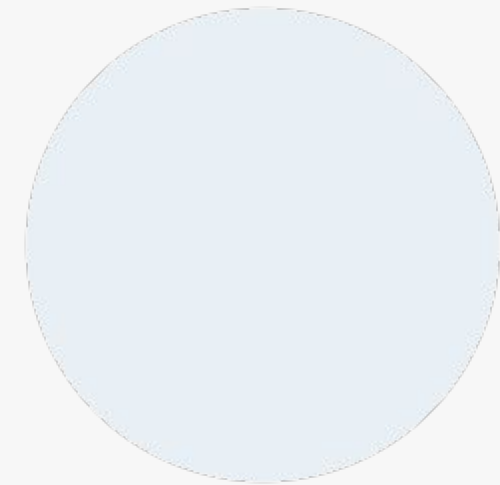


ASTON.



Selenium. Часть 2



astondevs.ru

Класс actions



Драйвер позволяет выполнять гораздо более интересные действия, чем просто клик или ввод символов. Для этих целей есть специальный класс Actions, который реализует паттерн «Строитель» для создания комплексных действий, включающих в себя несколько простых.

Рассмотрим пример выделения нескольких строк таблицы. Будучи опытными пользователями, такие действия мы иногда делаем на автомате. Но если разложить процесс на составляющие действия, то получим примерно следующее:

- Кликнуть левой кнопкой мыши на первую строку таблицы.
- Зажать CTRL.
- Кликнуть левой кнопкой мыши на другую строку таблицы.
- Отпустить клавишу CTRL.

Посмотрим, как выглядит этот пример в коде:

```
// Создадим экземпляр класса Actions
Actions builder = new Actions(driver);
// Построим цепочку действий
builder
    .click(tableRows.get(0)) // кликаем на первую
                             // строку
    .keyDown(Keys.CONTROL) // зажимаем CTRL
    .click(tableRows.get(2)) // кликаем на другую
                             // (третью) строку
    .keyUp(Keys.CONTROL) // отпускаем клавишу
    .build() // завершаем формирование цепочки
             // действий
    .perform(); // отдаем команду на исполнение
```

Класс actions



При создании любой цепочки действий построение завершается методом build. Метод perform — это спусковой крючок, после вызова которого собранное действие выполняется.

Класс Keys, также использованный в примере выше, служит для представления не буквенно-цифровых клавиш клавиатуры: CTRL, SHIFT, FN, F1 и т. д.

Рассмотрим основных представителей класса, не задействованных в учебном примере:

```
// Отправка нажатий в текстовое поле
Actions sendKeys(CharSequence... keys)
// Клик с удержанием
Actions clickAndHold(WebElement target)
// Отпустить зажатую клавишу мыши
Actions release()
// Двойной клик
Actions doubleClick(WebElement target)
// Наведение фокуса на элемент
Actions moveToElement(WebElement target)
Actions moveToElement(WebElement target, int xOffset, int yOffset)
// Перемещение фокуса, указывая смещение относительно верхнего левого угла
Actions moveByOffset(int xOffset, int yOffset)
// Клик правой кнопкой мыши
Actions contextClick(WebElement target)
Actions contextClick()
// Перетаскивание объекта
Actions dragAndDrop(WebElement source, WebElement target)
Actions dragAndDropBy(WebElement source, int xOffset, int yOffset)
// Пауза
Actions pause(Duration duration)
```

Тип возвращаемого значения всех методов — Actions. Возвращение своего типа в методах характерно для классов, реализующих паттерн «Строитель».

JS Executor



WebDriver имеет довольно богатый API, предоставляющий удобный способ взаимодействия с веб-страницами в коде тестов. Тем не менее есть вещи, которые он делать не умеет. Иногда мы упираемся в ограничение реализации драйвера для конкретного браузера. Например, когда новый инструментарий из стандарта WebDriver ещё не выполнен. Порой API не предоставляет способа получить интересующую нас информацию. Для таких случаев припасено секретное оружие — JavaScript.

Интерфейс JavaScriptExecutor представляет собой контракт из двух методов — `executeScript` и `executeAsyncScript`. Эти методы отличаются, как это видно из названия, способом выполнения кода — последовательное и асинхронное. Они (методы) возвращают экземпляр класса `Object`, что обязывает тестировщика делать явное приведение типов в конкретных случаях.

Все распространённые реализации WebDriver имплементируют этот интерфейс, так что для использования достаточно привести экземпляр `driver` к требуемому типу:

```
JavaScriptExecutor jsExecutor = (JavaScriptExecutor)
driver;
long windowWidth = (long)
jsExecutor.executeScript("return
window.innerWidth");
```

JS Executor



Если присмотреться к сигнатурам методов (к слову, аналогичных), то на примере вышеупомянутого метода увидим следующее:

```
Object executeScript(String script, Object... args);
```

Первым аргументом метод принимает строковое представление js-кода, на втором месте — аргументы, передаваемые в скрипт. Конструкция VarArgs не обязывает пользователя передавать что-либо в случае отсутствия аргументов, поэтому на все случаи жизни нам хватает одного метода без лишних перегрузок.

```
private boolean isImageLoaded(WebElement image) {  
    JavascriptExecutor jsExecutor =  
    (JavascriptExecutor) driver;  
    return (boolean) jsExecutor.executeScript(  
        "return arguments[0].complete && " +  
        "typeof arguments[0].naturalWidth != 'undefined'  
&& " +  
        "arguments[0].naturalWidth > 0", image);  
}
```


JS Executor



Приведённый выше пример иллюстрирует пример использование JavaScript в коде тестов для выполнения проверок с передачей аргумента в скрипт. Как видно из листинга метода, передавая экземпляр `image` класса `WebElement`, в скрипте не фигурирует название переменной, вместо него — указание индекса в массиве аргументов `arguments[0]`.

JavaScriptExecutor не требуется для возвращения значения из веб-страницы. Он полезен, когда надо создать новую вкладку браузера, сделать кастомный скролл внутри страницы или кастомного элемента (например, карусели), взаимодействовать с уведомлениями и так далее. Причина в том, что любой современный браузер содержит встроенный интерпретатор JavaScript. Использование этой фичи ограничено, по сути, только фантазией.

Работа с окнами и фреймами



Зачастую процесс тестирования происходит в рамках одной веб-страницы. Но API веб-драйвера позволяет покрыть самые разные потребности пользователей, предоставляя удобные способы переключения контекста на вложенные фреймы, алерт-диалоги и даже другие вкладки браузера. Если изучить исходный код интерфейса WebDriver, можно найти метод с названием `switchTo`, что дословно переводится, как «переключись на...». Рекомендуется изучать исходные коды используемых библиотек, так как все важные методы имеют говорящие названия.

Такой метод возвращает объект, имплементирующий интерфейс `TargetLocator`, который включает следующие методы:

```
WebDriver frame(int index);
```

```
WebDriver frame(String nameOrId);
```

```
WebDriver frame(WebElement frameElement);
```

```
WebDriver parentFrame();
```

```
WebDriver window(String nameOrHandle);
```

```
WebDriver defaultContent();
```

```
WebElement activeElement();
```

```
Alert alert();
```

Работа с окнами браузера



Начнём с окон. Судя по названиям, из всех сигнатур вышеперечисленных методов только один метод позволяет переключить окно — `window`. На вход метод ожидает получить строковое значение с так называемым дескриптором окна — своеобразным ID, по которому драйвер отдаёт распоряжение браузеру на переключение окна. А где взять этот ID? Для этого есть два важных метода:

```
Set<String> getWindowHandles(); // возвращает ID
всех окон
String getWindowHandle(); // возвращает ID текущего
окна
```

Итак, чтобы переключить на нужное окно, требуется знать его дескриптор. Итоговая команда выглядит примерно так:

```
// Получение списка идентификаторов окон
List<String> windowHandles = new
ArrayList(driver.getWindowHandles());
// Получение конкретного идентификатора
String secondTab = windowHandles.get(1);
// Переключение
driver.switchTo().window(secondTab);
```


Работа с фреймами



Хотя для пользователя отдельный фрейм выглядит всего лишь чужаковатым элементом интерфейса веб-страницы, обычно их нет в современных сайтах, фреймы используют собственные DOM-деревья. И это не удивительно, ведь, по сути, фрейм представляет собой веб-страницу внутри веб-страницы. В связи с чем веб-драйверу, который ориентируется на DOM, требуется явно передать намерение к использованию другого DOM-дерева.

Возвращаясь к перечню методов интерфейса TargetLocator, можно заметить метод `frame` и две его перегрузки, предоставляющие следующий инструментарий:

- переключение в другой фрейм по его индексу;
- переключение в другой фрейм по его атрибуту `name` или `id`;
- переключение в другой фрейм, обёрнутый в объект `WebElement`.

Работа с фреймами



Логика действий не отличается от того, как надо работать с окнами, не считая, что драйвер не умеет возвращать все id или name фреймов на странице. Но так как фреймы обозначаются тегами <frame> и <iframe>, поиск концептуально не отличается от поиска любого другого элемента.

Пример:

```
// Переключение фокуса в первый найденный фрейм  
driver.switchTo().frame(0);
```

Если требуется возврат, переключение в родительский фрейм или корень страницы происходит с применением этих методов:

```
driver.switchTo().parentFrame()
```

Или:

```
driver.switchTo().defaultContent()
```

Практическое задание



Продолжим работу над блоком «Онлайн пополнение без комиссии» сайта mts.by.

1. Проверить надписи в незаполненных полях каждого варианта оплаты услуг: услуги связи, домашний интернет, рассрочка, задолженность;
2. Для варианта «Услуги связи» заполнить поля в соответствии с пререквизитами из предыдущей темы, нажать кнопку «Продолжить» и в появившемся окне проверить корректность отображения суммы (в том числе на кнопке), номера телефона, а также надписей в незаполненных полях для ввода реквизитов карты, наличие иконок платёжных систем.